

ADVANCED WEB DEVELOPMENT FRAMEWORKS (ITUE301)

Practical 5: MongoDB Integration and Schema Design with Mongoose	
CO/PO	CO2, CO3 / PO3, PO5
Objective	To connect a MongoDB database to an Express server and enforce data validation through Mongoose schema.
Prerequisites	• Practical 4 completed working Express server with in-memory CRUD • Basic understanding of NoSQL/document-based data (vs relational tables)
Theory Concepts (Self-Study Reference)	
Coursera Pointer	See Week 5 IBM: Node.js & MongoDB Developing Back-end Database Applications, Module 2 (data and databases intro, MongoDB CRUD operations, Mongoose schema design and validation).
Architecture / Diagram	
Diagram	<pre> graph TD ExpressApp[Express App] --> V1[] V1 --> MongooseODM[Mongoose ODM] MongooseODM --> V2[] V2 --> MongoDBDatabase[MongoDB Database] MongoDBDatabase --> tasksCollection[tasks collection] tasksCollection --> schema["{ title, description, completed, createdAt }"] </pre> Schema Validation: required fields + default values enforced before any document reaches the database.
Problem Definition	
Problem Statement	• Extend the Task Management backend from Practical 4 by connecting MongoDB using Mongoose. • Define a Task schema with at least 4 fields: title (String, required), description (String), completed (Boolean, default false), createdAt (Date, default Date.now). • Replace the in-memory array from Practical 4 with real Mongoose model operations. • Test all CRUD operations against the live database using Postman. • Ensure validation errors are returned as structured JSON, not raw Mongoose error objects.
Key Questions / Analysis	• What is the purpose of a schema in a NoSQL database like MongoDB, given that MongoDB itself is schema-less? • Why is it important to define required fields and default values at the schema level rather than relying on frontend validation alone? • What happens internally when a document fails Mongoose validation where is the request stopped?
Supplementary Problems	• Add a priority field to the schema restricted to an enum of ['low', 'medium', 'high']. • Add a pre-save hook that automatically trims whitespace from the title field. • Implement a GET /tasks/:id endpoint that returns a 404 JSON response if the ID does not exist.

Key Skills Addressed	• Connecting a Node/Express app to MongoDB using Mongoose • Designing a schema with field types, required constraints, and defaults • Performing CRUD operations using Mongoose model methods • Returning structured validation error responses
Learning Outcome	Students will be able to connect an Express application to MongoDB using Mongoose, define a validated schema, and perform real CRUD operations against a live database.
Lab Session Step-by-Step	
Steps	1. Install Mongoose and create a .env file for the connection string: <pre>npm install mongoose dotenv</pre> 2. Connect to MongoDB inside server.js: <pre>const mongoose = require('mongoose'); require('dotenv').config(); mongoose.connect(process.env.MONGO_URI) .then(() => console.log('MongoDB connected')) .catch((err) => console.error(err));</pre> 3. Define the Task schema and model in a separate models/Task.js file: <pre>const taskSchema = new mongoose.Schema({ title: { type: String, required: true }, description: { type: String }, completed: { type: Boolean, default: false }, createdAt: { type: Date, default: Date.now } }); module.exports = mongoose.model('Task', taskSchema);</pre> 4. Replace the in-memory array logic from Practical 4 with Mongoose methods: Task.find(), Task.create(), Task.findByIdAndUpdate(), Task.findByIdAndDelete(). 5. Wrap each route in a try/catch and pass errors to the global error handler using next(err). 6. Test all 4 endpoints in Postman; confirm data persists in MongoDB after server restart.
Tools / Technology / Resources Required	• Node.js (v18+), npm • Express.js, Mongoose • MongoDB (local or Atlas) • Postman or Thunder Client
Total Hours	• Lab Session: 2 hours • Implementation: 1.5 hours • Testing / Modification: 0.5 hours • Total Engagement: 2 hours
Faculty Teaching Guide	
Guidance Notes	• Decide in advance: local MongoDB install vs MongoDB Atlas free tier demonstrate connection string setup for whichever is chosen. • Show the MongoDB Compass GUI (or Atlas web UI) so students can visually confirm documents are being created. • Emphasize that schema validation happens at the Mongoose layer, not in the database itself important for understanding NoSQL flexibility vs application-level enforcement. • Walk through one validation failure live so students see the actual error shape before they handle it.
Common Mistakes	• Hardcoding the MongoDB connection string directly in code instead of using a .env file. • Forgetting required: true on fields that should never be empty, allowing invalid documents to be saved. • Not wrapping database calls in try/catch, causing unhandled promise rejections to crash the server. • Sending the raw Mongoose ValidationError object to the client instead of a clean, structured message.

Troubleshooting Guide		
Symptom	Likely Cause	Fix
MongoDB connection times out	Incorrect connection string or IP not whitelisted (Atlas)	Verify MONGO_URI in .env; if using Atlas, whitelist the lab's IP address or allow access from anywhere for lab use
Validation error not caught	Mongoose operation not wrapped in try/catch	Wrap all async Mongoose calls in try/catch and forward errors using next(err)
Data not persisting after restart	Still using in-memory array instead of Mongoose model	Confirm all CRUD routes use Task.find/create/update/delete, not the old array
Cannot find module 'dotenv'	Package not installed	Run npm install dotenv and ensure require('dotenv').config() is called at the top of server.js
Student Assignment / Post Laboratory Work		
Tasks	• Review the Theory Concepts section and the assigned Coursera module before next week. • Add a priority field to the schema restricted to an enum of low/medium/high. • Implement a GET /tasks/:id endpoint with proper 404 handling.	
GitHub Deliverables		
Expected Repository Contents	• Same repository as Practical 4, updated with new commits • Working MongoDB-backed CRUD API with Mongoose schema and validation • .env excluded via .gitignore; .env.example provided instead • At least one commit specifically for MongoDB/Mongoose integration	
Rubrics (Practical Evaluation / Viva)		
Criteria (Marks)	Description	
Conceptual understanding (2 Marks)	Explains what and why, not just what was done	
Implementation (4 Marks)	Code written, runs, and is original	
Correctness (2 Marks)	Output matches expected result	
Reflection (1 marks)	Errors documented and explained	
Lab file evidence (1 marks)	Completeness and presentation	
Total: 10 marks	Passing Threshold: 5 / 10 (50%)	